FH CAMPUS WIEN
UNIVERSITY OF APPLIED SCIENCES

Designdokument

Geschwindigkeitsabhängiges Schutzsystem

Safety- und Security-Demonstrator mit rotierendem Aktor

Gruppe 3

Andreas Günter Kostecka
Jonas Gök
Loïc Thill
David Freudenthaler

FH Campus Wien

Stand: 27. Oktober 2026

Inhaltsverzeichnis

1 Projektbeschreibung **2**
1.1 Ziel und Systemidee . . . 2
1.2 Systemaufbau . . . 2
1.3 Funktionsablauf . . . 3
1.4 Zentrale Designparameter . . . 3
1.5 Grenzen des Demonstrators . . . 4

2 Softwarearchitektur und Design **5**
2.1 Raspberry Pi 1 – Host und Bedienung . . . 5
2.2 Raspberry Pi 2 – Steuerung und Überwachung . . . 5
2.3 Safety-Logik-Thread . . . 5
2.4 Verwendete Abstände . . . 6
2.5 Plausibilitätsprüfung der Sensorwerte . . . 6
2.6 Systemzustände . . . 7
2.7 Gemeinsam verwendete Daten . . . 8
2.8 Bluetooth-Kommunikation . . . 9
2.9 Motorsteuerung . . . 10
2.10 Anzeige und Logging . . . 10
2.11 Initialisierung und kontrolliertes Beenden . . . 11
2.12 Programmierrichtlinien und Versionskontrolle . . . 11
2.13 Pinout . . . 11

3 Programmablauf und Safety-Logik **13**

4 Safety Hazards **15**
4.1 Übersicht . . . 15
4.2 Adressierte Hazards . . . 15
4.3 Nicht vollständig adressierte Hazards . . . 17

5 Bedrohungen **19**
5.1 Übersicht . . . 20
5.2 Adressierte Bedrohungen . . . 20
5.3 Nicht vollständig adressierte Bedrohungen . . . 22
5.4 Abgrenzung . . . 23

6 Anforderungen und Tests **24**
6.1 Safety-Anforderungen . . . 24
6.2 Security-Anforderungen . . . 25
6.3 Unterstützende Anforderungen . . . 25
6.4 Geplante Tests . . . 26

1 Projektbeschreibung

1.1 Ziel und Systemidee

Ziel des Projekts ist ein Demonstrator für ein geschwindigkeitsabhängiges Schutzsystem eines rotierenden Aktors. Das Prinzip orientiert sich vereinfacht an *Speed and Separation Monitoring* (SSM): Mit steigender Geschwindigkeitsstufe wird ein größerer Sicherheitsabstand verwendet. Unterschreitet ein Objekt diesen Abstand, wird die Motoransteuerung abgeschaltet.

Der Demonstrator dient ausschließlich zur Untersuchung eines sicherheitsgerichteten Prinzips. Er ist kein zertifiziertes Safety-System. Insbesondere werden Raspberry Pi 4B, HC-SR04, Bluetooth und der verwendete Motortreiber nicht als Safety-zertifizierte Komponenten betrachtet.

Sicherer Zustand: Motor AUS – der Rotor wird nicht aktiv angetrieben.

Ein Software-Stopp bedeutet nicht automatisch mechanischen Stillstand. Der Rotor kann aufgrund seiner Massenträgheit nachlaufen. Diese Nachlaufzeit muss bei der Festlegung der Sicherheitsabstände berücksichtigt werden.

1.2 Systemaufbau

Das System besteht aus zwei Raspberry Pi 4B, zwei Ultraschall-Abstandssensoren HC-SR04, einem Elektromotor mit Rotor, einem Motortreiber, einem LCD-Display und einer Bluetooth-Verbindung zwischen den Raspberry Pis.

Komponente	**Aufgabe**	**Bedeutung für Safety**
Raspberry Pi 1	Bedienung, Auswahl der Geschwindigkeitsstufe, LCD-Anzeige, Logging und Bluetooth-Kommunikation	Liefert Sollwerte und zeigt Zustände an. LCD und Logging sind keine Safety-Funktionen.
Raspberry Pi 2	Sensorerfassung, Plausibilitätsprüfung, Safety-Logik und Motorsteuerung	Führt die lokale Safety-Entscheidung aus und kann den Motor unabhängig von Pi 1 abschalten.
2 × HC-SR04	Abstandsmessung zum Gefahrenbereich	Zwei gleiche Sensoren ermöglichen einen Quervergleich, bilden aber keine nachgewiesene Safety-Redundanz.
Motortreiber / Motor	Umsetzung der Motoransteuerung und rotierende Gefahrenquelle	Motor AUS entspricht PWM = 0 und deaktiviertem Enable-Signal. Mechanischer Stillstand ist dadurch nicht garantiert.
Bluetooth	Bidirektionale Kommunikation zwischen Pi 1 und Pi 2	Wird mit Heartbeat, Timeout und Plausibilitätsprüfungen überwacht.

Tabelle 1: Komponenten und Aufgaben des Demonstrators

1.3 Funktionsablauf

Nach dem Start initialisieren beide Raspberry Pis ihre benötigten Schnittstellen und Threads. Raspberry Pi 2 hält die Motoransteuerung während der Initialisierung im sicheren Zustand. Erst wenn gültige Sensordaten, eine gültige Kommunikation und eine gültige Geschwindigkeitsstufe vorliegen, darf der Motor freigegeben werden.

Raspberry Pi 1 sendet die gewünschte Geschwindigkeitsstufe `LOW`, `MEDIUM` oder `HIGH` sowie einen Heartbeat an Raspberry Pi 2. Raspberry Pi 2 liest beide Abstandssensoren aus, prüft die Messwerte und wählt den zugehörigen Sicherheitsabstand. Befindet sich ein Objekt innerhalb dieses Abstands oder schlägt eine Safety-Prüfung fehl, wird der Motor abgeschaltet.

Raspberry Pi 2 sendet Status, Abstandswerte und Fehlermeldungen an Raspberry Pi 1 zurück. Diese Informationen werden am LCD angezeigt und protokolliert.

Der Motor läuft wieder an, sobald in einem Zyklus des Safety-Logik-Threads alle Prüfungen bestanden sind und kein Objekt innerhalb des Sicherheitsabstands erkannt wird (Wiederanlaufbedingung, siehe Abschnitt 2.6 und SAF-11). Eine Quittierung durch den Bediener ist nicht vorgesehen.

1.4 Zentrale Designparameter

Die folgenden Werte sind Projektparameter für den Demonstrator und müssen am realen Aufbau getestet und gegebenenfalls angepasst werden.

Parameter	**Startwert**	**Bedeutung**
Heartbeat Pi 1 → Pi 2	100 ms (10 Hz)	Zyklische Lebensmeldung und Übertragung der Sollstufe
Kommunikations-Timeout	300 ms	Keine gültige Nachricht innerhalb dieses Zeitraums führt zu Motor AUS
Safety-Logik-Zyklus	50 ms	Zyklische Prüfung der Safety-Bedingungen auf Pi 2
Max. Alter der Sensordaten	150 ms	Ältere Messwerte dürfen nicht für eine Freigabe verwendet werden
Gültiger Messbereich	2–400 cm	Werte außerhalb des Messbereichs des HC-SR04 sind ungültig
Max. Abstandsänderung	0,2 cm/ms (= 2 m/s)	Schnellere Änderung zwischen zwei Messungen eines Sensors ist unplausibel
Sensorabweichung	$>$ 5 cm in zwei aufeinanderfolgenden Messzyklen	Sensorwerte werden als unplausibel bewertet
Sicherheitsabstände	$S_{\text{LOW}} < S_{\text{MEDIUM}} < S_{\text{HIGH}}$	Endgültige Werte werden am Demonstrator anhand von Reaktions- und Nachlaufzeit festgelegt

Tabelle 2: Zentrale Startwerte des Designs

1.5 Grenzen des Demonstrators

Die tatsächliche Motordrehzahl wird nicht mit einem Encoder gemessen. Daher darf nicht angenommen werden, dass ein PWM-Wert proportional zur realen Drehzahl ist. Die Geschwindigkeitsstufen stellen lediglich definierte Ansteuerungsklassen dar. Außerdem können zwei gleiche HC-SR04 durch eine gemeinsame Fehlerursache ähnliche falsche Werte liefern. Ein Defekt des Motortreibers, bei dem der Motor trotz Abschaltsignal weiter bestromt wird, kann mit der vorhandenen Architektur nicht sicher beherrscht werden.

Der Wiederanlauf erfolgt automatisch, ohne Quittierung durch den Bediener. Nach einem Fehler oder einem Objekt-Stopp kann der Motor daher wieder anlaufen, ohne dass der Bediener dies ausdrücklich freigegeben hat. Befindet sich ein Objekt genau an der Grenze des Sicherheitsabstands, kann der Motor wiederholt ein- und ausschalten. Diese Einschränkungen werden für den Demonstrator bewusst akzeptiert. In einer industriellen Anwendung wäre ein unerwarteter Wiederanlauf durch eine Quittierung und eine Hysterese zu verhindern.

2 Softwarearchitektur und Design

Die Software wird in C unter Linux auf beiden Raspberry Pis implementiert. Threads werden verwendet, um Sensorerfassung, Kommunikation, Bedienung und Safety-Logik voneinander zu trennen. Safety-relevante Entscheidungen werden auf Raspberry Pi 2 getroffen; ein Ausfall von LCD oder Logging darf die Motorabschaltung nicht verhindern.

2.1 Raspberry Pi 1 – Host und Bedienung

Modul / Thread	**Aufgabe**
Initialisierung	Bluetooth, LCD, Logging und benötigte Threads starten
UI / Eingabe	Geschwindigkeitsstufe `LOW`, `MEDIUM`, `HIGH` auswählen und Programmende anfordern
Kommunikations-/Heartbeat-Thread	Sollstufe und Heartbeat alle 100 ms senden; Status und Fehlermeldungen von Pi 2 empfangen
Anzeige	Systemzustand, Geschwindigkeitsstufe, kleinsten gemessenen Abstand $d_{\min}$ und Fehlercode am LCD anzeigen (siehe Abschnitt 2.4)
Logging	Ereignisse sofort und Messwerte mit 10 Hz protokollieren (siehe Abschnitt 2.10); keine Safety-Funktion

Tabelle 3: Softwareaufgaben auf Raspberry Pi 1

2.2 Raspberry Pi 2 – Steuerung und Überwachung

Modul / Thread	**Aufgabe**
Initialisierung	Motor sicher deaktivieren, Sensoren, Bluetooth und Threads initialisieren
Sensor-Thread 1 / 2	HC-SR04 auslesen; Messbereich und Abstandsänderung prüfen; Messwert, Gültigkeitsstatus und Zeitstempel bereitstellen
Kommunikations-Thread	Nachrichten von Pi 1 empfangen und prüfen; Heartbeat-Timeout überwachen; Status alle 100 ms an Pi 1 senden
Safety-Logik-Thread	Zyklisch alle 50 ms Sensorplausibilität, Kommunikation und Datenaktualität prüfen; Sicherheitsabstand auswählen und Motorfreigabe entscheiden
Motorsteuerung	Motorstufe setzen oder Motor mit PWM = 0 und Enable = OFF abschalten

Tabelle 4: Softwareaufgaben auf Raspberry Pi 2

2.3 Safety-Logik-Thread

Der Safety-Logik-Thread ist ein eigener Thread auf Raspberry Pi 2. Er führt keine blockierenden Netzwerk- oder Logging-Aufrufe aus. In jedem Zyklus werden folgende Prüfungen in dieser Reihenfolge ausgeführt:

1. Sind beide Sensorwerte gültig und zueinander plausibel?

2. Ist die Kommunikation aktuell und der empfangene Befehl gültig?
3. Sind die verwendeten Sensordaten noch aktuell?
4. Welcher Sicherheitsabstand gehört zur gültigen Geschwindigkeitsstufe?
5. Liegt der aktuelle Abstand innerhalb des Sicherheitsabstands?

Schlägt eine der ersten drei Prüfungen fehl, wird ein Fehlerzustand gesetzt und die Motoransteuerung deaktiviert. Erst wenn alle Prüfungen erfolgreich sind, darf die normale Motorsteuerung ausgeführt werden.

2.4 Verwendete Abstände

Im Dokument werden mehrere Abstände verwendet. Sie haben unterschiedliche Aufgaben:

Größe	**Bedeutung**
d_1, d_2	Gemessene Abstände von Sensor 1 und Sensor 2. Beide werden benötigt, um die Sensoren gegeneinander zu prüfen (P3).
$d_{\min} = \min(d_1, d_2)$	Kleinerer der beiden gültigen Messwerte. Er wird für die Abschaltentscheidung verwendet, weil er den ungünstigeren Fall darstellt.
S_{LOW}, S_{MEDIUM}, S_{HIGH}	Sicherheitsabstand je Geschwindigkeitsstufe. Bei höherer Stufe sind Nachlaufzeit und zurückgelegter Weg des Rotors größer, deshalb gilt $S_{\text{LOW}} < S_{\text{MEDIUM}} < S_{\text{HIGH}}$.
S	Aktiver Sicherheitsabstand, also der Wert, der zur aktuell gültigen Geschwindigkeitsstufe gehört. Der Motor wird abgeschaltet, wenn $d_{\min} < S$.

Tabelle 5: Begriffe der Abstandsüberwachung

Am LCD wird nur $d_{\min}$ angezeigt. d_1 und d_2 werden einzeln protokolliert, damit Abweichungen zwischen den Sensoren nachvollziehbar sind.

2.5 Plausibilitätsprüfung der Sensorwerte

Die Abweichung der beiden Sensoren ist nicht die einzige Prüfung. Ein Sensorwert wird nur verwendet, wenn er alle folgenden Prüfungen besteht. Die Prüfungen P1 und P2 betreffen jeden Sensor einzeln und werden im jeweiligen Sensor-Thread durchgeführt. Das Ergebnis wird als Gültigkeitsstatus im Datensatz abgelegt. P3 und P4 vergleichen bzw. bewerten die Daten und werden im Safety-Logik-Thread durchgeführt.

ID	Prüfung	Bedingung für „ungültig“	Ort
P1	Messbereich	$d < 2$ cm oder $d > 400$ cm (Grenzen des HC-SR04). Ein fehlendes Echo führt zu einem Wert außerhalb des Messbereichs und wird dadurch ebenfalls erkannt.	Sensor-Thread
P2	Abstandsänderung	$\frac{\lvert d_k - d_{k-1}\rvert}{t_k - t_{k-1}} > 0{,}2$ cm/ms, also eine Änderung schneller als 2 m/s zwischen zwei Messungen desselben Sensors.	Sensor-Thread
P3	Quervergleich	$\lvert d_1 - d_2\rvert > 5$ cm in zwei aufeinanderfolgenden Messzyklen.	Safety-Logik
P4	Datenalter	Messwert älter als 150 ms.	Safety-Logik

Tabelle 6: Plausibilitätsprüfungen der Abstandssensoren

Die Grenze von 2 m/s in P2 liegt über der Annäherungsgeschwindigkeit von 1,6 m/s, die in der EN ISO 13855 für Körperbewegungen angesetzt wird, und filtert vor allem Ausreißer und Sprünge des Messwerts. Taucht ein Objekt seitlich im Messbereich auf, kann der Messwert ebenfalls sprunghaft kleiner werden und P2 auslösen. Die Reaktion darauf ist in beiden Fällen Motor AUS und liegt damit auf der sicheren Seite.

2.6 Systemzustände

Raspberry Pi 2 befindet sich immer in genau einem der folgenden Zustände. Der aktuelle Zustand wird im Status an Pi 1 übertragen und am LCD angezeigt.

Zustand	Motor	Bedeutung
`INIT`	AUS	Programmstart; Schnittstellen und Threads werden initialisiert. Noch keine Safety-Bewertung.
`BETRIEB`	AN (`LOW`, `MEDIUM` oder `HIGH`)	Alle Prüfungen bestanden und $d_{\min} \geq S$.
`OBJEKT_STOPP`	AUS	Alle Prüfungen bestanden, aber $d_{\min} < S$. Kein Fehler, sondern bestimmungsgemäße Schutzreaktion.
`FEHLER`	AUS	Mindestens eine Prüfung fehlgeschlagen. Ein Fehlercode gibt die Ursache an: `SENSOR` (P1–P3), `KOMMUNIKATION` (Timeout oder ungültiger Befehl), `DATENALTER` (P4).
`SHUTDOWN`	AUS	Stop/Shutdown-Befehl empfangen; Threads werden kontrolliert beendet. Endzustand.

Tabelle 7: Systemzustände von Raspberry Pi 2

`INIT` wird nach Abschluss der Initialisierung verlassen. Danach wird der Zustand in jedem Zyklus des Safety-Logik-Threads neu aus den Prüfergebnissen bestimmt, in derselben Reihenfolge wie im Flussdiagramm (Abbildung 3):

1. Stop/Shutdown empfangen → `SHUTDOWN`

2. Eine der Prüfungen Sensorplausibilität, Kommunikation oder Datenaktualität fehlgeschlagen → `FEHLER`
3. $d_{\min} < S \rightarrow$ `OBJEKT_STOPP`
4. sonst → `BETRIEB`

Wiederanlaufbedingung: Der Fehlerzustand wird nicht gespeichert. Aus `FEHLER` oder `OBJEKT_STOPP` geht das System in `BETRIEB` über, sobald in einem Zyklus alle folgenden Bedingungen gleichzeitig erfüllt sind:

- kein Stop/Shutdown-Befehl empfangen,
- beide Sensorwerte gültig und zueinander plausibel (P1–P3),
- gültige Nachricht von Pi 1 innerhalb der letzten 300 ms mit gültiger Geschwindigkeitsstufe,
- Sensordaten nicht älter als 150 ms (P4),
- $d_{\min} \geq S$ für die aktuell gültige Stufe.

Der Motor läuft dann spätestens im nächsten Zyklus, also nach höchstens 50 ms, mit der von Pi 1 angeforderten Stufe an. Eine Quittierung durch den Bediener ist nicht vorgesehen; die daraus folgenden Einschränkungen sind in Abschnitt 1.5 beschrieben.

Auf Pi 1 wird zusätzlich „Verbindung getrennt“ angezeigt, wenn länger als 300 ms kein gültiger Status von Pi 2 empfangen wurde. Der zuletzt empfangene Zustand wird dann nicht mehr als aktuell dargestellt.

2.7 Gemeinsam verwendete Daten

Sensor- und Kommunikations-Threads aktualisieren Daten, die vom Safety-Logik-Thread gelesen werden. Damit keine Race Conditions entstehen, werden zusammengehörige Werte als Datensatz behandelt, zum Beispiel:

```
struct sensor_data {
    float distance_cm;
    bool valid;
    struct timespec timestamp;
};
```

Der schreibende Thread aktualisiert den vollständigen Datensatz geschützt durch einen Mutex. Der Safety-Logik-Thread kopiert den Datensatz unter demselben Mutex in eine lokale Variable und arbeitet anschließend nur mit diesem Snapshot. Zeitmessungen für Timeouts und Datenalter sollen mit einer monotonen Uhr erfolgen.

Konkret werden folgende gemeinsamen Datensätze verwendet:

Datensatz	Schreibt	Liest	Inhalt
`sensor_data[2]`	Sensor-Thread 1 / 2	Safety-Logik, Kommunikation	Abstand, Gültigkeit (P1, P2), Zeitstempel
`comm_data`	Kommunikations-Thread	Safety-Logik	Sollstufe, Stop-Flag, Zeitstempel der letzten gültigen Nachricht
`safety_state`	Safety-Logik	Kommunikation, Motorsteuerung	Systemzustand, Fehlercode, freigegebene Motorstufe

Tabelle 8: Gemeinsame Datensätze auf Raspberry Pi 2

Für die Umsetzung gelten folgende Regeln:

- Jeder Datensatz hat genau einen schreibenden Thread und einen eigenen `pthread_mutex_t`.
- Die Mutexe werden mit dem Protokoll `PTHREAD_PRIO_INHERIT` angelegt, damit ein niedriger priorisierter Thread den Safety-Logik-Thread nicht durch Prioritätsinversion aufhält.
- Unter einem Mutex wird nur kopiert. Messungen, Bluetooth-Zugriffe, Logging und GPIO-Zugriffe erfolgen außerhalb des gesperrten Bereichs.
- Ein Thread hält immer nur einen Mutex gleichzeitig; dadurch sind Deadlocks ausgeschlossen.
- Der Safety-Logik-Thread läuft mit `SCHED_FIFO` und der höchsten Priorität der Anwendung.
- Nur der Safety-Logik-Thread schreibt die freigegebene Motorstufe. Die Motorsteuerung setzt ausschließlich diesen Wert um.

2.8 Bluetooth-Kommunikation

Die Kommunikation ist bidirektional. Raspberry Pi 1 sendet Sollstufe, Heartbeat, Sequenznummer und gegebenenfalls einen Stop/Shutdown-Befehl. Raspberry Pi 2 sendet Status, aktuelle Abstandswerte und Fehlermeldungen zurück.

Nachrichten werden mindestens auf Nachrichtentyp, Länge, Wertebereich und Sequenznummer geprüft. Alte, doppelte oder unzulässige Befehle werden nicht als gültige Motorfreigabe verwendet. Bleibt eine gültige Nachricht länger als 300 ms aus, wird die Kommunikation als fehlerhaft bewertet und der Motor abgeschaltet.

Umsetzung der Datenübertragung:

- Die Verbindung wird als RFCOMM-Socket über die BlueZ-Socket-API aufgebaut (`AF_BLUETOOTH`, `SOCK_STREAM`, `BTPROTO_RFCOMM`). Pi 2 ist Server und wartet auf einem festen Kanal, Pi 1 ist Client. Die beiden Geräte werden einmalig gekoppelt.
- Da RFCOMM einen Datenstrom und keine einzelnen Pakete liefert, beginnt jede Nachricht mit einem Typ- und einem Längenfeld. Der Empfänger liest zuerst diese Felder und danach genau die angegebene Anzahl Bytes.

- Nachrichten werden Feld für Feld in ein Byte-Array mit fester Byte-Reihenfolge geschrieben, nicht als C-`struct` kopiert. Dadurch spielen Padding und Compiler-Unterschiede keine Rolle.
- Abstände werden als ganze Millimeter (`uint16_t`) übertragen, Stufen, Zustände und Fehlercodes als `uint8_t` mit festgelegten Werten. Unbekannte Werte führen zum Verwerfen der Nachricht.
- Der Kommunikations-Thread wartet mit `poll()` und einem Timeout von 50 ms auf Daten. Er blockiert dadurch nie dauerhaft und kann den Heartbeat-Timeout auch dann auswerten, wenn keine Daten ankommen.
- Bricht die Verbindung ab, versucht Pi 1 zyklisch, sie neu aufzubauen. Pi 2 bleibt währenddessen über den Kommunikations-Timeout im Zustand `FEHLER`.

Nachricht	**Richtung**	**Takt**	**Nutzdaten**
`HEARTBEAT`	Pi 1 → Pi 2	100 ms	Sollstufe
`STOP`	Pi 1 → Pi 2	bei Bedarf	–
`STATUS`	Pi 2 → Pi 1	100 ms	Systemzustand, Fehlercode, aktive Stufe, d_1, d_2
`ACK`	Pi 2 → Pi 1	bei Bedarf	Bestätigung des kontrollierten Shutdowns

Tabelle 9: Nachrichtentypen zwischen Pi 1 und Pi 2

Jede Nachricht enthält zusätzlich die Sequenznummer und die kryptografische Absicherung gemäß Abschnitt 5.

2.9 Motorsteuerung

Die Motorsteuerung erhält ausschließlich eine bereits durch die Safety-Logik freigegebene Geschwindigkeitsstufe. Eine unbekannte oder ungültige Stufe führt nicht zu einem Default-Betrieb, sondern zu Motor AUS. Der sichere Softwarezustand wird durch

`PWM = 0` und `Enable = OFF`

angefordert. Der mechanische Nachlauf des Rotors wird dadurch nicht aufgehoben.

2.10 Anzeige und Logging

Anzeige und Logging laufen auf Pi 1 und sind keine Safety-Funktionen. Das LCD zeigt Systemzustand, Geschwindigkeitsstufe, $d_{\min}$ und gegebenenfalls den Fehlercode an und wird bei jedem empfangenen Status aktualisiert.

Protokolliert wird auf zwei Arten:

- **Ereignisse** werden sofort beim Auftreten protokolliert: Zustandswechsel, neuer Fehlercode, Änderung der Sollstufe durch den Bediener, Verbindungsverlust und Wiederaufbau, Programmstart und Stop/Shutdown.

- **Messwerte** werden mit 10 Hz protokolliert, also bei jeder empfangenen `STATUS`-Nachricht: Zeitstempel, Zustand, Fehlercode, Stufe, d_1 und d_2.

Das Log wird als CSV-Datei geschrieben. Die Ausgabe erfolgt gepuffert und wird einmal pro Sekunde sowie bei jedem Fehlerereignis auf die SD-Karte geschrieben, damit Logging die Bedienung und Kommunikation nicht verzögert.

2.11 Initialisierung und kontrolliertes Beenden

Beim Start von Raspberry Pi 2 ist die Motoransteuerung deaktiviert. Eine Motorfreigabe ist erst erlaubt, wenn Initialisierung, Sensorprüfung und Kommunikationsprüfung erfolgreich abgeschlossen wurden.

Fordert der Benutzer auf Raspberry Pi 1 das Programmende an, sendet Pi 1 einen Stop/Shutdown-Befehl. Pi 2 setzt zuerst den Motor in den sicheren Zustand, beendet danach seine Threads und schließt die Bluetooth-Verbindung. Anschließend kann Pi 1 seine eigenen Threads und Ressourcen kontrolliert beenden.

2.12 Programmierrichtlinien und Versionskontrolle

Der C-Code soll MISRA-C-Regeln soweit im Projekt praktikabel berücksichtigen. Insbesondere werden feste Datentypen, geprüfte Rückgabewerte, definierte Wertebereiche, ein `default`-Zweig in `switch`-Anweisungen und möglichst kurze, klar abgegrenzte Funktionen verwendet. Gemeinsame Konstanten wie Timeouts und Grenzwerte werden zentral definiert.

Das gesamte Projekt wird mit Git versioniert, nicht nur der Safety-Teil. Im Repository liegen:

- der Quellcode von Pi 1 und Pi 2,
- gemeinsame Header-Dateien (Nachrichtenformat, Konstanten und Grenzwerte),
- Build-Dateien und Testskripte,
- dieses Designdokument.

Ausgenommen ist nur das kryptografische Schlüsselmaterial, das über `.gitignore` vom Repository ausgeschlossen wird. Änderungen am Safety-relevanten Code auf Pi 2 werden über einen Pull Request eingebracht und vor dem Merge von einem zweiten Gruppenmitglied geprüft.

2.13 Pinout

Die folgenden Abbildungen zeigen die Pinbelegung der beiden Raspberry Pis.

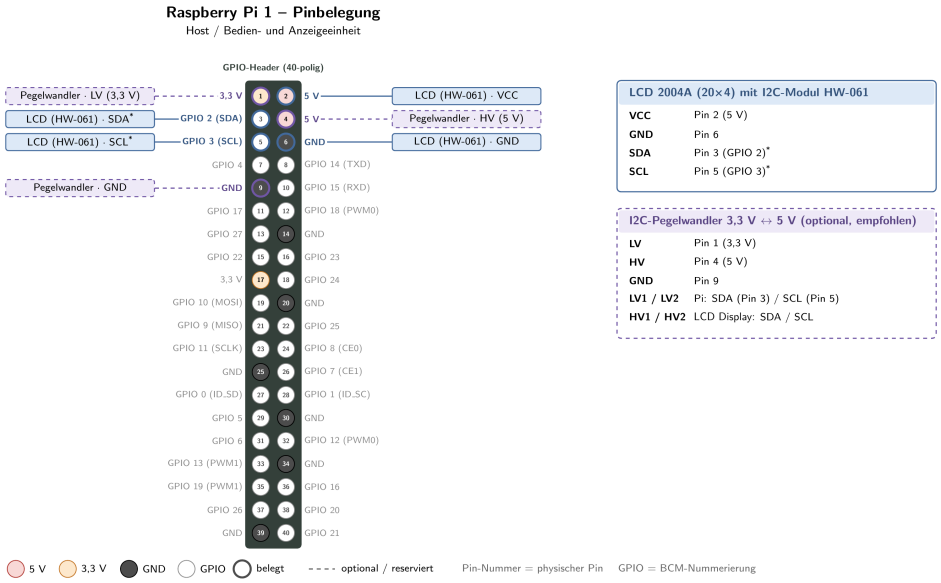


Abbildung 1: Pinbelegung von Raspberry Pi 1

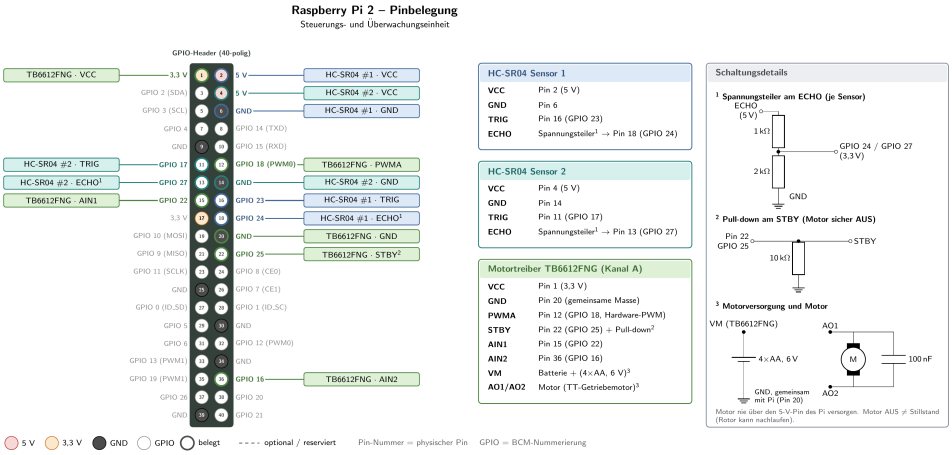


Abbildung 2: Pinbelegung von Raspberry Pi 2

Der Link zum Github-Repository lautet https://github.com/freudehcw/Safety_Strategies_Gruppe3

3 Programmablauf und Safety-Logik

Abbildung 3 zeigt den Ablauf des Gesamtsystems. Raspberry Pi 1 übernimmt Bedienung, Anzeige, Logging und die zyklische Heartbeat-Kommunikation. Raspberry Pi 2 führt die Sensorerfassung, Safety-Logik und Motorsteuerung aus.

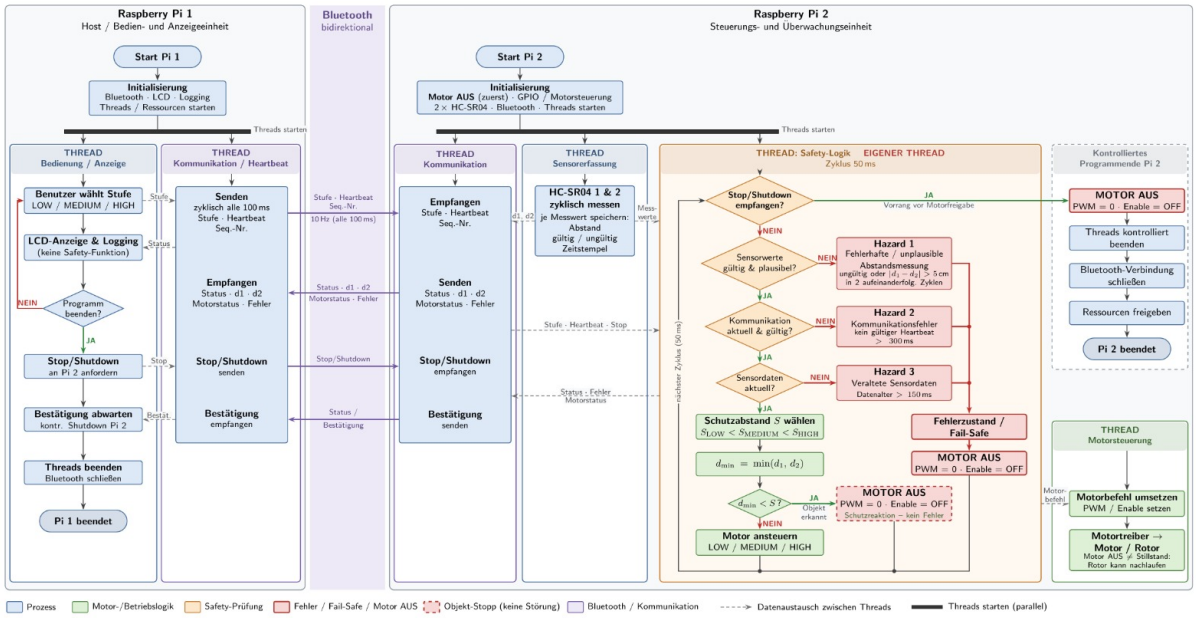


Abbildung 3: Flussdiagramm des Gesamtsystems mit den drei adressierten Safety Hazards

Der Safety-Logik-Thread auf Raspberry Pi 2 arbeitet zyklisch und prüft drei Freigabebedingungen. Die Prüfungen sind logisch UND-verknüpft: Nur wenn alle drei Bedingungen erfüllt sind, wird der Sicherheitsabstand ausgewertet und eine Motorfreigabe ist möglich.

1. **Sensorplausibilität:** Beide Sensorwerte müssen gültig sein. Eine Abweichung von mehr als 5 cm in zwei aufeinanderfolgenden Messzyklen wird als Fehler bewertet. Zusätzlich werden Messbereich und Abstandsänderung jedes Sensors geprüft (siehe Abschnitt 2.5).

2. **Kommunikation:** Eine gültige Nachricht bzw. ein Heartbeat muss rechtzeitig eintreffen. Bei mehr als 300 ms ohne gültige Nachricht wird der Motor abgeschaltet.

3. **Datenaktualität:** Sensordaten dürfen höchstens 150 ms alt sein. Ältere Daten gelten als ungültig.

Nach bestandenen Prüfungen wird der Sicherheitsabstand entsprechend der Geschwindigkeitsstufe ausgewählt. Ist der aktuelle Abstand kleiner als der Sicherheitsabstand, wird der Motor abgeschaltet. Andernfalls darf der Motor mit der freigegebenen Stufe betrieben werden.

Nach jedem Zyklus beginnt die Prüfung von vorn. Ein Fehler oder Objekt-Stopp wird daher nicht gespeichert: Sind im nächsten Zyklus alle Bedingungen erfüllt, läuft der Motor ohne Quittierung wieder an (Wiederanlaufbedingung, siehe Abschnitt 2.6).

Das Programmende ist ebenfalls Teil des Ablaufs. Ein Stop/Shutdown-Befehl von Raspberry Pi 1 führt auf Raspberry Pi 2 zuerst zu Motor AUS und anschließend zum kontrollierten Beenden des Programms.

4 Safety Hazards

Für den Demonstrator werden sechs zentrale Safety Hazards betrachtet. Drei davon werden durch konkrete Maßnahmen im Design adressiert. Drei weitere werden bewusst nicht vollständig adressiert, weil dafür zusätzliche unabhängige oder sicherheitsgerichtete Hardware erforderlich wäre.

Sicherer Zustand: Motor AUS – der Rotor wird nicht aktiv angetrieben.

4.1 Übersicht

ID	Status	Hazard	Umgang im Projekt
H1	Adressiert	Motor läuft weiter, obwohl ein Objekt zu nahe ist, weil ein Sensorwert falsch oder ungültig ist.	Messbereichs- und Änderungsratenprüfung je Sensor, Vergleich beider Sensoren (5-cm-Grenze); bei Fehler Motor AUS.
H2	Adressiert	Motor arbeitet mit einem veralteten oder ungültigen Geschwindigkeitsbefehl weiter.	10-Hz-Heartbeat, Sequenznummer und 300-ms-Timeout; bei Kommunikationsfehler Motor AUS.
H3	Adressiert	Motor läuft weiter, obwohl veraltete Sensordaten verwendet werden.	Zeitstempel je Messwert, maximales Datenalter 150 ms; bei Überschreitung Motor AUS.
H4	Nicht vollständig adressiert	Rotor läuft trotz angefordertem STOP weiter oder kommt zu spät zum Stillstand.	Software setzt PWM = 0 und Enable = OFF; Nachlauf wird berücksichtigt, Defekt der Leistungsstufe bleibt offen.
H5	Nicht vollständig adressiert	Tatsächliche Motordrehzahl ist höher als vom System angenommen.	Keine unabhängige Drehzahlmessung; konservative Geschwindigkeitsklassen und Schutzabstände.
H6	Nicht vollständig adressiert	Beide HC-SR04 liefern gleichzeitig ähnliche falsche Werte.	Quervergleich erkennt Common-Cause-Fehler nicht sicher; Grenze wird dokumentiert.

Tabelle 10: Zentrale Safety Hazards des Demonstrators

4.2 Adressierte Hazards

H1 – Fehlerhafte oder unplausible Abstandsmessung

Hazard: Der Motor läuft weiter, obwohl sich ein Objekt bereits innerhalb des erforderlichen Sicherheitsabstands befindet.

Ursache: Ein Sensor liefert keinen Wert, einen zu großen Wert oder einen deutlich anderen Wert als der zweite Sensor.

Erkennung und Maßnahme: Beide HC-SR04 müssen einen gültigen Messwert liefern. Jeder Messwert muss zunächst im Messbereich von 2–400 cm liegen (P1) und darf sich gegenüber der vorherigen Messung desselben Sensors nicht schneller als 0,2 cm/ms ändern (P2). Anschließend wird die absolute Differenz der beiden Werte geprüft (P3). Ist

$$|d_1 - d_2| > 5\,\text{cm}$$

in zwei aufeinanderfolgenden Messzyklen, gelten die Sensoren als unplausibel. Ein fehlender oder ungültiger Messwert wird ebenfalls als Fehler behandelt. Die vollständige Liste der Prüfungen steht in Abschnitt 2.5. Für die Abstandsentscheidung wird der kleinere der beiden gültigen Werte verwendet.

Fail-Safe-Reaktion:

Ungültige oder unplausible Sensordaten → Motor AUS

Test: Einen Sensor verdecken, gezielt einen abweichenden Messwert erzeugen oder per Testmodus Werte außerhalb des Messbereichs bzw. mit unplausiblem Sprung einspeisen. Der Fehler muss erkannt werden und der Motor muss abschalten.

H2 – Kommunikationsfehler zwischen Raspberry Pi 1 und Raspberry Pi 2

Hazard: Der Motor arbeitet mit einem alten oder ungültigen Geschwindigkeitsbefehl weiter.

Ursache: Bluetooth-Verbindung unterbrochen, Pi 1 ausgefallen, Nachricht verspätet oder Nachrichteninhalt ungültig.

Erkennung und Maßnahme: Pi 1 sendet alle 100 ms einen Heartbeat zusammen mit dem aktuellen Befehl. Pi 2 prüft Wertebereich und Sequenznummer. Trifft länger als 300 ms keine gültige Nachricht ein, wird ein Kommunikationsfehler gesetzt. Damit entsprechen drei ausbleibende Heartbeats dem Timeout.

Fail-Safe-Reaktion:

Kommunikationsfehler oder Timeout → Motor AUS

Test: Während des Motorbetriebs die Bluetooth-Verbindung trennen oder das Pi-1-Programm anhalten. Spätestens nach dem definierten Timeout muss Pi 2 den Motor abschalten.

H3 – Veraltete Sensordaten

Hazard: Die Safety-Entscheidung wird mit einem Abstandswert getroffen, der nicht mehr den aktuellen Zustand beschreibt.

Ursache: Ein Sensor-Thread hängt, wird verspätet ausgeführt oder aktualisiert seine Daten nicht mehr.

Erkennung und Maßnahme: Jeder Messwert wird mit einem monotonen Zeitstempel gespeichert. Vor der Verwendung prüft der Safety-Logik-Thread das Alter. Messwerte älter als 150 ms gelten als ungültig und dürfen keine Motorfreigabe bewirken.

Fail-Safe-Reaktion:

Sensordaten zu alt → Motor AUS

Test: Einen Sensor-Thread im Testmodus länger als 150 ms blockieren. Der Safety-Thread muss den veralteten Messwert erkennen und den Motor abschalten.

4.3 Nicht vollständig adressierte Hazards

H4 – Motor oder Motortreiber reagiert nicht korrekt auf STOP

Hazard: Der Rotor wird trotz angeforderter Abschaltung weiter angetrieben oder kommt erst verspätet zum Stillstand.

Warum relevant: Ein Software-Stoppkommando garantiert weder eine funktionierende Leistungselektronik noch sofortigen mechanischen Stillstand.

Warum nicht vollständig adressiert: Die vorhandene Architektur besitzt keinen unabhängigen sicherheitsgerichteten Abschaltpfad und keine Rückmeldung, ob der Rotor tatsächlich steht.

Maßnahme im Demonstrator: PWM und Enable werden deaktiviert. Die Nachlaufzeit des Rotors wird am realen Aufbau gemessen und bei der Wahl der Sicherheitsabstände berücksichtigt.

Industrielle Lösung: Unabhängige sicherheitsgerichtete Abschaltung, beispielsweise geeigneter Safety-Controller, Sicherheitsrelais oder ein Antrieb mit Safe-Torque-Off.

H5 – Tatsächliche Motordrehzahl ist unbekannt

Hazard: Der Rotor dreht schneller, als es die gewählte Geschwindigkeitsstufe vermuten lässt.

Warum relevant: PWM-Prozent und reale Motordrehzahl sind nicht gleichzusetzen. Last, Versorgungsspannung und Motoreigenschaften beeinflussen die tatsächliche Drehzahl und Nachlaufzeit.

Warum nicht vollständig adressiert: Es ist kein unabhängiger Encoder oder anderer Drehzahlsensor vorhanden.

Maßnahme im Demonstrator: Es werden nur die Klassen `LOW`, `MEDIUM` und `HIGH` verwendet. Für höhere Stufen wird ein größerer, konservativ festgelegter Sicherheitsabstand verwendet.

Industrielle Lösung: Unabhängige Drehzahlerfassung mit sicherheitsgerichteter Auswertung.

H6 – Gemeinsamer Fehler beider Abstandssensoren

Hazard: Beide HC-SR04 liefern gleichzeitig ähnliche, aber falsche Abstandswerte.

Warum relevant: Beide Sensoren verwenden dasselbe Messprinzip. Ungünstige Oberflächen, Winkel, Reflexionen oder gemeinsame Störungen können daher beide Sensoren ähnlich beeinflussen.

Warum nicht vollständig adressiert: Ein Vergleich zweier gleicher Sensoren erkennt nur Unterschiede zwischen den Messwerten. Sind beide Werte durch dieselbe Ursache falsch, kann die Plausibilitätsprüfung erfolgreich sein, obwohl der reale Abstand gefährlich klein ist.

Maßnahme im Demonstrator: Die Sensoren werden zeitlich versetzt angesteuert und die Einschränkung des Quervergleichs wird ausdrücklich dokumentiert.

Industrielle Lösung: Geeignete sicherheitsgerichtete Sensorik und je nach Anwendung diversitäre Messprinzipien.

5 Bedrohungen

Die Security-Betrachtung orientiert sich an den klassischen Schutzzielen der Informationssicherheit (CIA-Triade: Vertraulichkeit, Integrität, Verfügbarkeit) sowie der Authentizität. Der Fokus liegt auf der Kommunikation zwischen den beiden Raspberry Pis. Da es sich um ein sicherheitskritisches System handelt, gilt der Grundsatz: Security-Maßnahmen dürfen die Safety (Fail-Safe-Reaktion) nicht beeinträchtigen. Die lokale Abstandsauswertung auf dem zweiten Raspberry Pi (im Folgenden Pi 2 genannt) bleibt auch dann aktiv, wenn die Funkverbindung gestört oder unterbrochen wird.

Drei Bedrohungen werden durch konkrete, über den reinen Bluetooth-Standard hinausgehende Maßnahmen im Design adressiert. Drei weitere werden bewusst nicht vollständig adressiert, weil dafür zusätzliche Hardware- oder Plattformmaßnahmen (z. B. Secure Boot, diversitäre Sensorik) erforderlich wären.

5.1 Übersicht

ID	Bedrohung	Mögliche Auswirkung	Maßnahme im Demonstrator
T1	Unberechtigtes Gerät sendet Befehle (Spoofing)	Verlust der Integrität und Authentizität: Unerwünschte Motorbedienung	Absicherung auf Applikationsebene (AES-GCM); Pi 2 verifiziert kryptografisch den Absender und die Integrität der Nachricht.
T2	Alte Nachricht wird erneut verarbeitet (Replay)	Veralteter Befehl könnte erneut wirksam werden	Fortlaufende Sequenznummer, die kryptografisch (z. B. als AAD in AES-GCM) an die Nachricht gebunden ist.
T3	Bluetooth wird gestört oder fällt aus (Jamming)	Verlust der Verfügbarkeit: Pi 2 erhält keine aktuellen Befehle mehr	Heartbeat und 300-ms-Timeout; bei Kommunikationsverlust Motor AUS. Safety bleibt erhalten.
T4	Kompromittierter Sender (Pi 1) sendet manipulierte Befehle	Unerwünschte Geschwindigkeitsstufe, falsche Statusanzeige für Bediener	*Nicht vollständig adressiert.* Pi 2 entscheidet lokal über die Abschaltung; der Sicherheitsabstand gilt für jede Stufe.
T5	Manipulation von Software oder Konfiguration auf Pi 2	Grenzwerte oder Safety-Logik werden deaktiviert (Integritätsverlust)	*Nicht vollständig adressiert.* Konstanten zentral definiert und über Git versioniert; kein Schutz am laufenden Gerät (kein Secure Boot).
T6	Akustische Störung/Spoofing der Ultraschallsensoren	Sensoren liefern falsche Abstände (Integrität der physikalischen Ebene gestört)	*Nicht vollständig adressiert.* Zeitversetztes Triggern schützt vor Eigeninterferenz, aber nicht vor gezieltem, externem Signal-Spoofing.

Tabelle 11: Security-Bedrohungen und Gegenmaßnahmen

5.2 Adressierte Bedrohungen

T1 – Unberechtigtes Gerät sendet Befehle (Spoofing/Injection)

Bedrohung (Integrität & Authentizität): Ein fremdes Gerät baut eine Verbindung zu Raspberry Pi 2 auf und sendet unautorisierte Steuerbefehle.

Angriffsweg: Ein Angreifer umgeht die standardmäßigen Bluetooth-Sicherheitsmechanismen (z. B. durch MAC-Spoofing oder Schwachstellen im Bluetooth-Protokoll) und sendet selbst erzeugte Nachrichten.

Erkennung und Maßnahme: Um nicht allein auf die Sicherheit des Bluetooth-Stacks zu vertrauen, wird eine Absicherung auf Applikationsebene implementiert. Die Kommunikation wird mittels Authenticated Encryption with Associated Data (AEAD), konkret

AES-GCM, abgesichert. Angemessen für den Rahmen eines Lehrprojekts wird auf einen komplexen Schlüsselaustausch verzichtet. Es wird stattdessen ein statischer **256-Bit Pre-Shared Key (PSK)** implementiert, der auf beiden Raspberry Pis hinterlegt ist. Jede Nachricht wird entschlüsselt und ihre Authentizität (via MAC/Auth-Tag) geprüft. Zusätzlich werden nur gültige Befehle (LOW, MEDIUM, HIGH) akzeptiert.

Reaktion:

Fehlerhaftes Auth-Tag oder ungültiger Inhalt → Nachricht verwerfen, keine Motorfreigabe

Test: Nachrichten ohne gültige Verschlüsselung oder mit absichtlich verfälschtem Auth-Tag einspeisen. Die Nachricht muss von Pi 2 verworfen werden, der Motorzustand darf sich nicht ändern.

T2 – Wiedereinspielen alter Nachrichten (Replay-Angriff)

Bedrohung (Integrität): Eine zuvor kryptografisch gültige Nachricht wird abgefangen und später erneut gesendet.

Angriffsweg: Der Angreifer belauscht den Funkverkehr. Ohne den AES-Schlüssel zu kennen, sendet er eine exakte Kopie einer alten Motor-*HIGH*-Nachricht zu einem späteren Zeitpunkt, um den Motor ungewollt wieder zu starten.

Erkennung und Maßnahme: Jede Steuerungsnachricht von Pi 1 trägt eine fortlaufende **32-Bit-Sequenznummer (implementiert als `uint32_t`)**, die bei jedem Verbindungsaufbau mit dem Wert 1 initialisiert wird. Diese Nummer fließt als *Additional Authenticated Data* (AAD) oder direkt im Payload in die AES-GCM-Verschlüsselung ein. Pi 2 speichert die zuletzt akzeptierte Nummer und verwirft alle Nachrichten mit gleicher oder kleinerer Sequenznummer. Ein **Überlauf** der 32-Bit-Variable tritt bei einer Sende-Frequenz von 10 Hz erst nach über 13 Jahren Dauerbetrieb auf. Dieser theoretische Fall wird softwareseitig abgefangen: Erreicht die Sequenznummer ihren Maximalwert, erzwingt Pi 2 einen Verbindungsabbruch (Übergang in den sicheren Zustand).

Reaktion:

Alte oder doppelte Sequenznummer → Nachricht verwerfen; bleibt ein gültiger Heartbeat aus → Motor AUS

Test: Im Testmodus eine bereits verarbeitete, valide verschlüsselte Nachricht erneut senden. Sie muss aufgrund der veralteten Sequenznummer ignoriert werden.

T3 – Störung oder Ausfall der Bluetooth-Verbindung (Jamming)

Bedrohung (Verfügbarkeit): Raspberry Pi 2 erhält keine aktuellen Befehle mehr und könnte in einem potenziell gefährlichen Zustand verbleiben.

Angriffsweg: Gezielte Funkstörung im 2,4-GHz-Band (Jamming), wodurch die legitime Kommunikation überlagert wird (Denial-of-Service).

Erkennung und Maßnahme: Pi 1 sendet alle 100 ms einen kryptografisch abgesicherten Heartbeat. Pi 2 überwacht den Zeitpunkt der letzten gültigen Nachricht. Hierfür wird in C eine **monotone Uhr** (mittels der POSIX-Funktion `clock_gettime(CLOCK_MONOTONIC,`

`...`)) verwendet. Diese zählt ausschließlich die Hardware-Zeit seit dem Systemstart und ist immun gegen Zeitsprünge, die bei normalen Systemuhren durch NTP-Synchronisation auftreten könnten. Trifft länger als 300 ms keine Nachricht mit korrektem Auth-Tag und neuer Sequenznummer ein, wird ein Kommunikationsfehler gesetzt. Die lokale Abstandsauswertung bleibt davon unabhängig bestehen.

Reaktion:

Kein gültiger Heartbeat innerhalb von 300 ms → Motor AUS

Ein Angreifer kann somit zwar die Verfügbarkeit (Availability) des Demonstrators zerstören, das System geht jedoch in einen sicheren Zustand über (Fail-Safe).

Test: Um den Ausfall der Funkschnittstelle realitätsnah zu simulieren, ohne Pi 1 physisch abzuschalten, wird auf Pi 1 die Bluetooth-Schnittstelle während des laufenden Betriebs per Software blockiert (z. B. via Kommandozeile mit `rfkill block bluetooth`). Spätestens nach 300 ms muss Pi 2 den Motor stoppen.

5.3 Nicht vollständig adressierte Bedrohungen

T4 – Kompromittierter Sender (Raspberry Pi 1)

Bedrohung (Integrität am Ursprung): Ein Angreifer erlangt Kontrolle über das Betriebssystem von Pi 1. Er besitzt damit auch Zugriff auf das gültige Schlüsselmaterial für AES-GCM.

Warum relevant: Die Applikationsverschlüsselung stellt nur sicher, dass eine Nachricht von Pi 1 stammt und auf dem Weg nicht verändert wurde. Sie kann nicht verifizieren, ob der auf Pi 1 generierte Befehl tatsächlich dem Willen des menschlichen Bedieners entspricht.

Warum nicht vollständig adressiert: Es gibt keine Integritätsprüfung der Laufzeitumgebung auf Pi 1 und keine unabhängige Bestätigung direkt auf Pi 2.

Maßnahme im Demonstrator: Die Abstandsüberwachung läuft völlig autonom auf Pi 2. Auch bei einem kompromittierten *HIGH*-Befehl schaltet Pi 2 den Motor ab, sobald der Sensor ein Hindernis erkennt.

Industrielle Lösung: Gehärtetes Betriebssystem (RTOS), Ausführung in einer Trusted Execution Environment (TEE) sowie eine unabhängige, hardwareverdrahtete Not-Halt-Einrichtung direkt an der Maschine.

T5 – Manipulation von Software oder Konfiguration auf Pi 2

Bedrohung (Integrität des Edge-Devices): Durch physischen Zugriff (z. B. Austausch der SD-Karte) werden Programm, kryptografische Schlüssel oder Safety-Parameter von Pi 2 verändert.

Warum relevant: Pi 2 trägt die gesamte Safety-Entscheidung. Eine Manipulation der Abstands-Schwellwerte hebelt das gesamte Konzept aus.

Warum nicht vollständig adressiert: Der verwendete Raspberry Pi bietet im Projektaufbau keine Hardware-Root-of-Trust und keinen Schutz des Speichermediums (SD-Karte

unverschlüsselt). Ein Angreifer mit physischem Zugriff liegt gemäß Abschnitt 5.4 außerhalb des Scope.

Maßnahme im Demonstrator: Parameter sind zentral im Code definiert und über Git versioniert (Nachvollziehbarkeit des Soll-Zustands).

Industrielle Lösung: Secure Boot mit signierter Firmware (z. B. mittels TPM-Modul), Full-Disk-Encryption und Zugriffsschutz am Gehäuse.

T6 – Akustische Störung oder Spoofing der Ultraschallsensoren

Bedrohung (Signal-Spoofing): Ein Angreifer beschallt die Sensoren gezielt mit 40 kHz-Ultraschallsignalen, um künstliche Echos zu erzeugen oder das echte Echo zu überlagern.

Warum relevant: Ein Angreifer greift hier nicht die IT-Schicht an, sondern die *physikalische Ebene* (Physical Layer Security). Ein manipulierter Abstandswert könnte dazu führen, dass der Motor trotz Hindernis nicht stoppt.

Warum nicht vollständig adressiert: Beide Sensoren nutzen das exakt gleiche Messprinzip (Common-Cause-Anfälligkeit). Ein intelligenter Acoustic-Spoofing-Angriff kann beide Sensoren zeitgleich so manipulieren, dass sie ähnliche falsche Werte liefern. Die Plausibilitätsprüfung (Vergleich der beiden Werte) wird dann bestanden und erkennt die Manipulation nicht.

Maßnahme im Demonstrator: Das zeitversetzte Triggern beider Sensoren verhindert Eigeninterferenzen. Die Verwendung des kleineren der beiden Messwerte bietet einen Grundschutz gegen einfache Störungen.

Industrielle Lösung: Diversitäre Sensorik (Kombination von Ultraschall mit Radar, Lidar oder optischen Lichtvorhängen) sowie codierte/modulierte Ultraschallsignale, bei denen der Sensor nur sein eigenes Echo an einem spezifischen Muster erkennt.

5.4 Abgrenzung

Der Demonstrator implementiert Mechanismen auf Applikationsebene (AES-GCM), um Sicherheitslücken des reinen Bluetooth-Standards zu mitigieren. Ein vollständiger Schutz gegen einen System-Kompromiss (Root-Rechte auf dem Betriebssystem) oder ein physisches Ersetzen von Komponenten liegt jedoch außerhalb des Projektumfangs. Entscheidend für das integrierte Konzept ist: Kein erfolgreicher IT-Angriff über die Funkschnittstelle darf dazu führen, dass die lokale Safety-Funktion von Pi 2 ausgehebelt wird.

6 Anforderungen und Tests

Die Anforderungen sind bewusst auf die für den Demonstrator wesentlichen Punkte beschränkt. Zeit- und Grenzwerte sind Design-Startwerte und müssen am realen Aufbau verifiziert werden.

6.1 Safety-Anforderungen

ID	Anforderung	Bezug
SAF-01	Der sichere Zustand des Systems ist Motor AUS. Auf Pi 2 bedeutet dies PWM = 0 und Enable = OFF.	H1–H6
SAF-02	Pi 2 muss beim Programmstart im sicheren Zustand bleiben, bis Initialisierung, Sensorprüfung und Kommunikation gültig sind.	H1–H3
SAF-03	Beide Abstandssensoren müssen gültige Werte liefern. Ein Messwert außerhalb von 2–400 cm oder mit einer Änderung von mehr als 0,2 cm/ms gegenüber der vorherigen Messung ist ungültig. Eine Abweichung der beiden Sensoren größer 5 cm in zwei aufeinanderfolgenden Messzyklen muss zu Motor AUS führen.	H1
SAF-04	Jeder Sensormesswert muss einen Zeitstempel besitzen. Daten älter als 150 ms dürfen keine Motorfreigabe bewirken.	H3
SAF-05	Pi 1 muss alle 100 ms einen Heartbeat senden. Erhält Pi 2 länger als 300 ms keine gültige Nachricht, muss der Motor abgeschaltet werden.	H2
SAF-06	Der Safety-Logik-Thread auf Pi 2 muss zyklisch alle 50 ms laufen und die Safety-Bedingungen vor einer Motorfreigabe prüfen.	H1–H3
SAF-07	Nur die gültigen Geschwindigkeitsstufen LOW, MEDIUM und HIGH dürfen verwendet werden. Für jede Stufe muss ein definierter Sicherheitsabstand existieren.	H5
SAF-08	Unterschreitet der aktuelle gültige Abstand den Sicherheitsabstand der gewählten Stufe, muss der Motor abgeschaltet werden.	H1, H5
SAF-09	Ein Stop/Shutdown-Befehl muss auf Pi 2 zuerst Motor AUS auslösen. Erst danach dürfen Threads und Programm beendet werden.	H4
SAF-10	Die endgültigen Sicherheitsabstände müssen am realen Aufbau anhand der beobachteten Reaktions- und Nachlaufzeiten festgelegt und dokumentiert werden.	H4, H5
SAF-11	Der Motor darf nach Motor AUS (Zustand `FEHLER` oder `OBJEKT_STOPP`) nur wieder anlaufen, wenn in einem Zyklus alle Bedingungen gleichzeitig erfüllt sind: kein Stop/Shutdown-Befehl, beide Sensorwerte gültig und plausibel, gültige Nachricht mit gültiger Stufe innerhalb der letzten 300 ms, Sensordaten nicht älter als 150 ms und $d_{\min} \geq S$. Der Wiederanlauf erfolgt ohne Quittierung (siehe Abschnitt 2.6).	H1–H3, H5

Tabelle 12: Safety-Anforderungen

6.2 Security-Anforderungen

ID	Anforderung
SEC-01	**Vertraulichkeit & Authentizität:** Die Kommunikation muss auf Applikationsebene kryptografisch abgesichert sein (z. B. AES-GCM), um unberechtigtes Mitlesen und Spoofing zu verhindern.
SEC-02	**Integrität:** Eingehende Nachrichten müssen vor der Verwendung zwingend kryptografisch verifiziert (z. B. Prüfung des Auth-Tags) und anschließend auf korrekten Typ, Länge und gültigen Wertebereich geprüft werden.
SEC-03	**Replay-Schutz:** Jede Steuerungsnachricht muss eine Sequenznummer tragen, die kryptografisch an den Payload gebunden ist (z. B. als AAD). Alte oder doppelte Nachrichten dürfen nicht akzeptiert werden.
SEC-04	**Verfügbarkeit vs. Safety:** Ein Kommunikationsangriff (z. B. Jamming) oder Verbindungsausfall darf die lokale Abstandsauswertung auf Pi 2 nicht deaktivieren; bei einem Timeout (siehe SAF-05) geht das System in den sicheren Zustand über (Motor AUS).

Tabelle 13: Security-Anforderungen

6.3 Unterstützende Anforderungen

- Das LCD zeigt den Systemzustand gemäß Abschnitt 2.6 (`INIT`, `BETRIEB`, `OBJEKT_STOPP`, `FEHLER` mit Fehlercode, `SHUTDOWN`), die Geschwindigkeitsstufe und den kleinsten gemessenen Abstand $d_{\min}$ an. Die Einzelwerte d_1 und d_2 werden nur protokolliert (siehe Abschnitt 2.4). Das LCD ist keine Safety-Funktion.

- Logging protokolliert Ereignisse (Zustandswechsel, Fehler, Stufenwechsel, Verbindungsverlust, Start und Ende) sofort und Messwerte mit 10 Hz, also bei jeder Statusnachricht (siehe Abschnitt 2.10). Logging ist keine Risikominderung und darf den Safety-Logik-Thread nicht blockieren.

- Gemeinsame Daten zwischen Threads werden je Datensatz durch einen eigenen Mutex mit Prioritätsvererbung geschützt; der Safety-Logik-Thread arbeitet nur mit einer Snapshot-Kopie. Die Bluetooth-Übertragung erfolgt über einen RFCOMM-Socket mit festem Nachrichtenformat (siehe Abschnitte 2.7 und 2.8).

- Das gesamte Projekt (Code beider Pis, gemeinsame Header, Build-Dateien, Testskripte und Dokumentation) wird über Git versioniert; nur Schlüsselmaterial ist ausgenommen. Änderungen an Safety-relevanter Software müssen vor dem Merge von einem zweiten Gruppenmitglied überprüft werden.

6.4 Geplante Tests

Test	Durchführung	Erwartetes Ergebnis
TC-01	Einen Sensor verdecken oder ungültigen Messwert erzeugen.	Sensorfehler wird erkannt; Motor AUS.
TC-01a	Im Testmodus einen Messwert außerhalb von 2–400 cm einspeisen.	Messwert wird als ungültig erkannt (P1); Motor AUS.
TC-01b	Im Testmodus einen Sprung des Messwerts um mehr als 0,2 cm/ms einspeisen.	Messwert wird als unplausibel erkannt (P2); Motor AUS.
TC-02	Zwei Sensorwerte um mehr als 5 cm abweichen lassen.	Nach zwei aufeinanderfolgenden unplausiblen Messzyklen Motor AUS.
TC-03	Bluetooth-Verbindung während des Motorbetriebs trennen oder Pi 1 anhalten.	Spätestens nach 300 ms ohne gültige Nachricht wird die Motorfreigabe entzogen.
TC-04	Sensor-Thread länger als 150 ms blockieren.	Daten werden als veraltet erkannt; Motor AUS.
TC-05	Testobjekt in den Sicherheitsabstand der jeweiligen Geschwindigkeitsstufe bewegen.	Motor wird abgeschaltet, sobald der gültige Abstand kleiner als der konfigurierte Sicherheitsabstand ist.
TC-06	Programmende auf Pi 1 auslösen.	Pi 1 sendet Stop/Shutdown; Pi 2 setzt zuerst Motor AUS und beide Programme beenden kontrolliert.
TC-09	Nach TC-05 das Testobjekt aus dem Sicherheitsabstand entfernen; nach TC-03 die Verbindung wiederherstellen.	Motor läuft erst wieder an, wenn alle Bedingungen aus SAF-11 erfüllt sind, und dann ohne Quittierung mit der angeforderten Stufe.
TC-07 *(Security)*	**Spoofing-Test:** Eine Nachricht mit ungültigem kryptografischen Auth-Tag oder manipuliertem Payload einspeisen.	Die Nachricht wird auf Applikationsebene verworfen. Der Systemzustand bleibt unverändert.
TC-08 *(Security)*	**Replay-Test:** Eine exakte, gültig verschlüsselte Kopie einer älteren Nachricht (veraltete Sequenznummer) erneut einspeisen.	Die Nachricht wird aufgrund der Sequenznummern-Prüfung verworfen. Motor schaltet nach Timeout ab (falls kein neuer Heartbeat folgt).

Tabelle 14: Geplante System-, Safety- und Security-Tests